

S3 BUCKET TASK

1. Create an S3 bucket and upload some objects to S3.

Objective

To create an Amazon S3 bucket and upload files/objects into the bucket.

Steps Performed

Logged in to the AWS Management Console.

Navigated to Amazon S3 → Buckets.

Created a new S3 bucket with the name: karthik-s3-demo-2026-001

Selected the AWS Region: US East (N. Virginia) – us-east-1

Kept the default S3 settings and created the bucket.

Opened the newly created bucket.

Clicked Upload → Add files.

Selected the required files and uploaded them to the S3 bucket.

Verified that the uploaded objects were available under the Objects section.

Bucket Name

karthik-s3-demo-2026-001

Uploaded Objects

html

test.txt

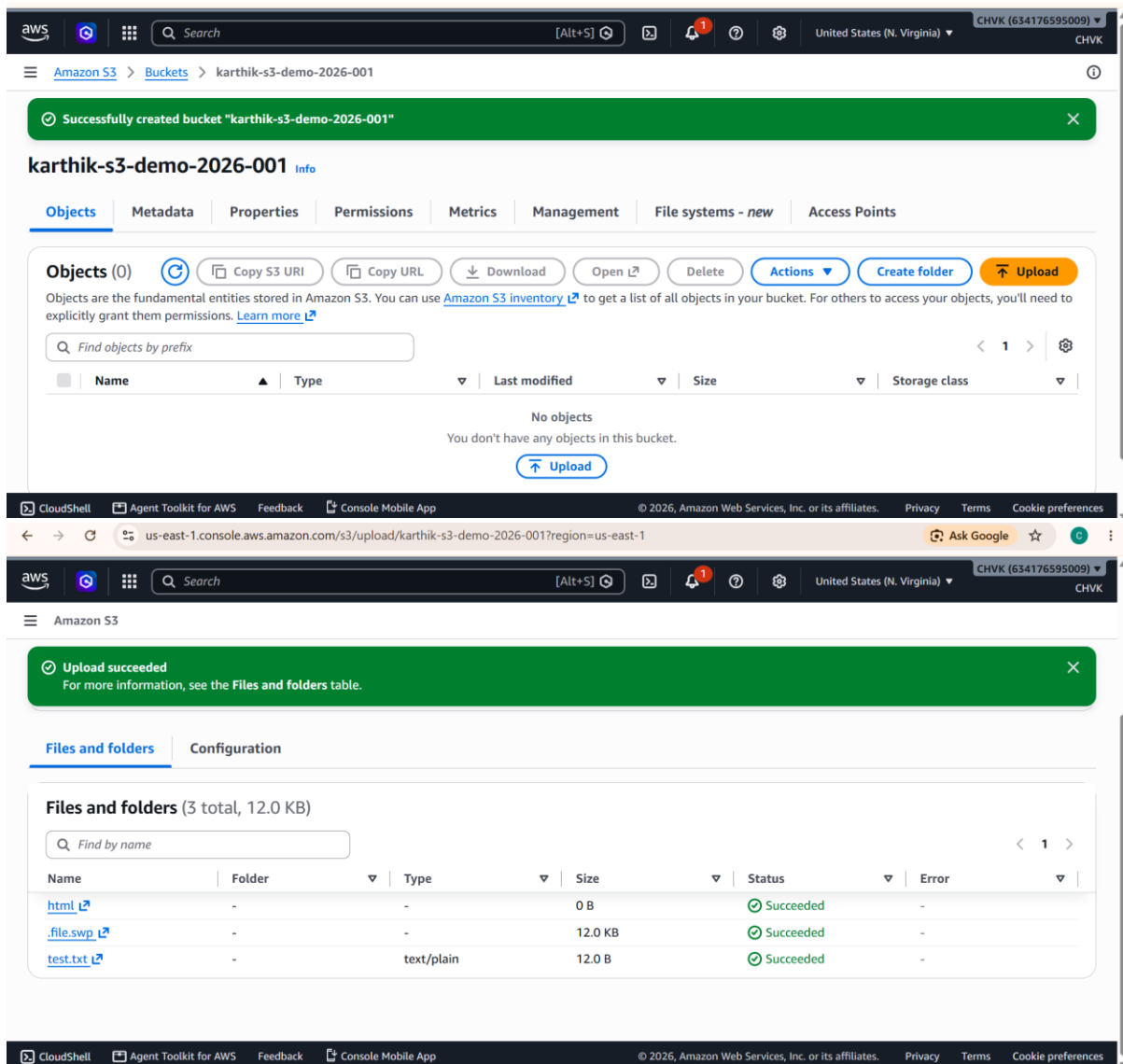
Result

The S3 bucket was successfully created, and the required objects were successfully uploaded and verified in the bucket.

Evidence

Screenshot 1: S3 bucket creation.

Screenshot 2: Uploaded objects displayed inside the S3 bucket.



2. Deploy a static website in the S3 bucket.

Objective

To deploy and host a static website using an Amazon S3 bucket.

Step 1: Create the HTML File

Created an index.html file on the Desktop using Notepad.

index.html Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Karthik S3 Website</title>
</head>
<body>
  <h1>Welcome to My S3 Static Website</h1>
  <p>This website is hosted using Amazon S3.</p>
</body>
```

</html>

The file was saved with the name:

index.html

Step 2: Upload the HTML File to S3

Opened the AWS Management Console.

Navigated to Amazon S3 → Buckets.

Opened the bucket:

karthik-s3-demo-2026-001

Clicked Upload → Add files.

Selected the index.html file from the Desktop.

Clicked Upload.

Verified that index.html was available under the Objects section.

Step 3: Enable Static Website Hosting

Opened the S3 bucket

Selected the Properties tab.

Navigated to Static website hosting.

Clicked Edit.

Enabled Static website hosting.

Selected Host a static website.

Entered index.html as the Index document.

Clicked Save changes.

Step 4: Configure Public Access

Navigated to the Permissions tab.

Opened Block public access (bucket settings).

Disabled Block all public access for this website bucket.

Confirmed the change.

Step 5: Configure Bucket Policy

Added the following bucket policy to allow public read access to the website objects:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::karthik-s3-demo-2026-001/*"
    }
  ]
}
```

}

Step 6: Test the Website

Navigated to Properties → Static website hosting.

Located the Bucket website endpoint.

Opened the endpoint in a web browser.

Verified that the website was displayed successfully.

The website displayed:

Welcome to My S3 Static Website

This website is hosted using Amazon S3.

Result

The static website was successfully deployed and hosted using Amazon S3. The website was successfully accessed through the S3 website endpoint.

Evidence / Screenshots

index.html file created.

index.html uploaded to the S3 bucket.

Static Website Hosting enabled.

Block Public Access configuration.

Bucket Policy configuration.

Static website successfully opened in the browser.

us-east-1.console.aws.amazon.com/s3/upload/karthik-s3-demo-2026-001?region=us-east-1

Ask Google

CHVK (634176595009)

aws

Search

[Alt+S]

1

United States (N. Virginia)

CHVK

Amazon S3

Upload succeeded

For more information, see the [Files and folders](#) table.

Destination

s3://karthik-s3-demo-2026-001

Succeeded

1 file, 203.0 B (100.00%)

Failed

0 files, 0 B (0%)

Files and folders

Configuration

Files and folders (1 total, 203.0 B)

Find by name

< 1 >

Name	Folder	Type	Size	Status	Error
index.html	-	text/html	203.0 B	Succeeded	-

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

us-east-1.console.aws.amazon.com/s3/buckets/karthik-s3-demo-2026-001?region=us-east-1&tab=properties

Ask Google

CHVK (634176595009)

aws

Search

[Alt+S]

Ask Amazon Q

1

United States (N. Virginia)

CHVK

Amazon S3 > Buckets > karthik-s3-demo-2026-001

Successfully edited static website hosting.

Static website hosting

Use this bucket to host a website or redirect requests. [Learn more](#)

We recommend using AWS Amplify Hosting for static website hosting

Deploy a fast, secure, and reliable website quickly with AWS Amplify Hosting. [Learn more about Amplify Hosting](#) or [View your existing Amplify apps](#)

Create Amplify app

S3 static website hosting

Enabled

Hosting type

Bucket hosting

Bucket website endpoint

When you configure your bucket as a static website, the website is available at the AWS Region-specific website endpoint of the bucket. [Learn more](#)

<http://karthik-s3-demo-2026-001.s3-website-us-east-1.amazonaws.com>

CloudShell

Agent Toolkit for AWS

Feedback

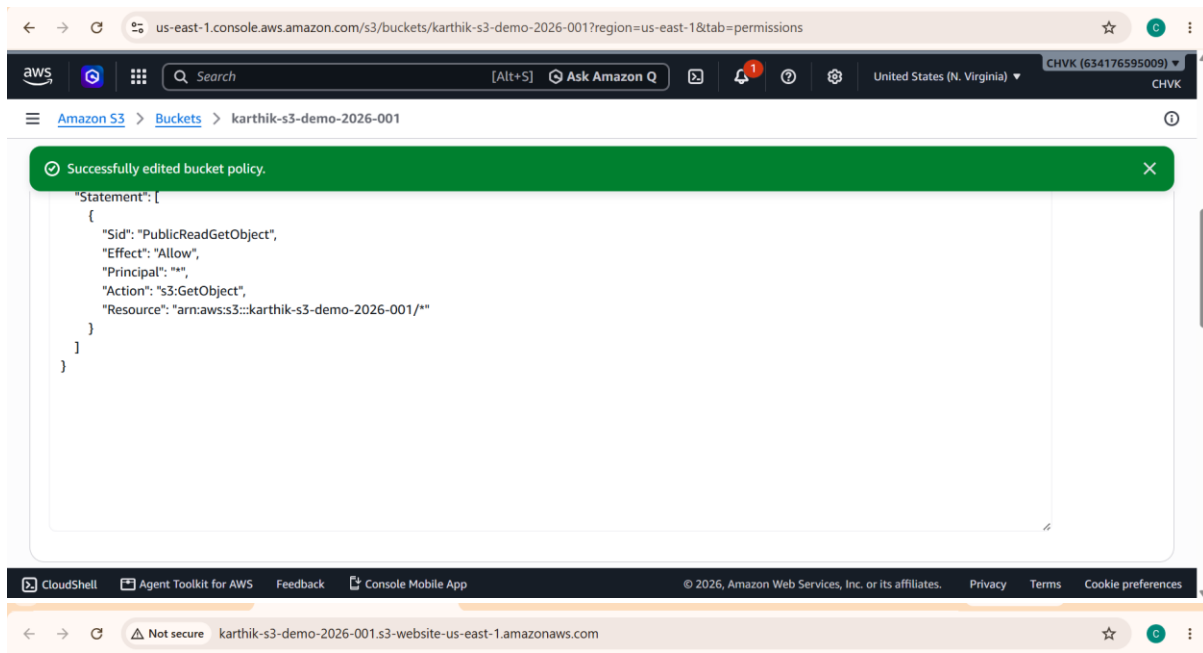
Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences



Welcome to My S3 Static Website

This website is hosted using Amazon S3.

3. Enable cross-region replication on S3 buckets.

Objective

To configure Amazon S3 Cross-Region Replication (CRR) between two S3 buckets in different AWS Regions and verify that objects uploaded to the source bucket are automatically replicated to the destination bucket.

Source Bucket

Bucket Name: karthik-s3-demo-2026-001

Region: US East (N. Virginia) – us-east-1

Destination Bucket

Bucket Name: karthik-s3-replica-2026-001

Region: US West (Oregon) – us-west-2

Steps Performed

1. Created Destination S3 Bucket

Created a second S3 bucket named:

karthik-s3-replica-2026-001

The bucket was created in the US West (Oregon) – us-west-2 Region.

2. Enabled Versioning

Enabled Bucket Versioning on both the source and destination buckets.

Source: karthik-s3-demo-2026-001

Destination: karthik-s3-replica-2026-001

Versioning is required for S3 replication.

3. Created Replication Rule

From the source bucket:

S3 → Source Bucket → Management → Replication rules → Create replication rule

Configured the following:

Replication Rule Name: replicate-to-us-west-2

Status: Enabled

Rule Scope: Entire bucket / All objects

Destination: karthik-s3-replica-2026-001

Destination Region: us-west-2

4. Configured IAM Role

Configured the S3 replication IAM role:

s3crr_role_for_karthik-s3-demo-2026-001

The trust policy was updated to allow both S3 and S3 Batch Operations to assume the role.

5. Tested Replication

Uploaded a new test object to the source bucket:

replication-test.txt.txt

The upload was successful in the source bucket.

After replication, the same object appeared in the destination bucket:

karthik-s3-replica-2026-001

This confirmed that Cross-Region Replication is working successfully.

Result

Successfully configured S3 Cross-Region Replication (CRR) from:

us-east-1 → us-west-2

The test object was successfully replicated from the source bucket to the destination bucket.

Evidence / Screenshots

Destination bucket created in us-west-2.

Versioning enabled on source and destination buckets.

Replication rule replicate-to-us-west-2 showing Enabled.

Test object uploaded to the source bucket.

replication-test.txt.txt successfully visible in the destination bucket.

us-west-2.console.aws.amazon.com/s3/buckets/karthik-s3-replica-2026-001?region=us-west-2&tab=objects

Ask Google

CHVK (634176595009)

aws

Search

[Alt+S]

United States (Oregon)

CHVK

Amazon S3

Buckets

karthik-s3-replica-2026-001

Successfully created bucket "karthik-s3-replica-2026-001"

karthik-s3-replica-2026-001

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Access

Objects (0)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name

Type

Last modified

Size

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

us-west-2.console.aws.amazon.com/s3/buckets/karthik-s3-replica-2026-001?region=us-west-2&tab=properties

Ask Google

CHVK (634176595009)

aws

Search

[Alt+S]

United States (Oregon)

CHVK

Amazon S3

Buckets

karthik-s3-replica-2026-001

Successfully edited Bucket Versioning

To transition, archive, or delete older object versions, [configure lifecycle rules](#) for this bucket.

AWS Region

US West (Oregon) us-west-2

Amazon Resource Name (ARN)

arn:aws:s3:::karthik-s3-replica-2026-001

Creation date

September 3, 2026, 16:35:59 (UTC+05:30)

Bucket Versioning

Edit

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

Enabled

Multi-factor authentication (MFA) delete

An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. [Learn more](#)

Disabled

Bucket properties

On the bucket Properties tab, you can edit bucket settings to support your use case.

Protect your storage

To save multiple distinct copies of your data and easily recover from both unintended user actions and application failures, you can enable Amazon S3 Versioning only for general purpose buckets. S3 Versioning is required to enable other data protection options for Amazon S3, including S3 Object Lock and S3 Replication.

Secure your storage

Amazon S3 applies server-side encryption with Amazon S3 managed keys (SSE-S3) or the

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

us-west-2.console.aws.amazon.com/s3/buckets/karthik-s3-replica-2026-001

Ask Google

CHVK (634176595009)

aws

Search

[Alt+S]

United States (Oregon)

CHVK

Amazon S3

Buckets

karthik-s3-replica-2026-001

Successfully edited Bucket Versioning

To transition, archive, or delete older object versions, [configure lifecycle rules](#) for this bucket.

Bucket overview

AWS Region

US West (Oregon) us-west-2

Amazon Resource Name (ARN)

arn:aws:s3:::karthik-s3-replica-2026-001

Creation date

September 3, 2026, 16:35:59 (UTC+05:30)

Bucket Versioning

Edit

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

Enabled

Multi-factor authentication (MFA) delete

An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To

Bucket properties

On the bucket Properties tab, you can edit bucket settings to support your use case.

Protect your storage

To save multiple distinct copies of your data and easily recover from both unintended user actions and application failures, you can enable Amazon S3 Versioning only for general purpose buckets. S3 Versioning is required to enable other data protection options for Amazon S3, including S3 Object Lock and S3 Replication.

Secure your storage

Amazon S3 applies server-side encryption with Amazon S3 managed keys (SSE-S3) or the

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

Batch Operations

Successfully created job ID 0e03eeb6-b9a9-4d92-a0c0-d657dcd112fa
The time it takes to prepare a job is based on the size of the job's manifest and the time required to complete higher-priority jobs.

Batch Operations Info
A job is used to execute batch operations on a list of S3 objects. Job events are published to [CloudWatch Events](#).

Jobs (1)

Job ID	Status	Description	Operation	Date created	Total objects	% Complete	Total failed (rate)	Priority
0e03eeb6-b9a9-4d92-a0c0-d657dcd112fa	New	2026-09-03 - Replicate	Replicate	September 3, 2026, 16:46:14 (UTC+05:30)	Not yet available	0%	0 (0%)	10

karthik-s3-replica-2026-001 Info

Objects Metadata Properties Permissions Metrics Management File systems - new Access Points

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Name	Type	Last modified	Size	Storage class
replication-test.txt.txt	txt	September 3, 2026, 16:58:45 (UTC+05:30)	26.0 B	Standard

4. Configure a bucket policy so only the Admin user can see the objects of the S3 bucket.

Objective

To create a private S3 bucket and configure a bucket policy so that only the Admin IAM user can list and read the objects in the bucket.

AWS Details

Bucket Name: karthik-s3-admin-only-2026-001-634176595009-us-west-2-an

Region: us-west-2 (US West/Oregon)

IAM User: Ec2-S3-Admin

IAM User ARN: arn:aws:iam::634176595009:user/Ec2-S3-Admin

Test Object: admin-test.txt.txt

Public Access: Block all public access enabled

Configuration

A separate private S3 bucket was created for this task because the bucket used for the static website in Task 2 was configured for public website access.

Object Ownership was kept as ACLs disabled and Block all public access was enabled.

A test object named admin-test.txt.txt was uploaded to the bucket.

The IAM user Ec2-S3-Admin was selected as the Admin user who should have access to the bucket.

Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AdminListBucket",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::634176595009:user/Ec2-S3-Admin"
      },
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::karthik-s3-admin-only-2026-001-634176595009-us-west-2-an"
    },
    {
      "Sid": "AdminReadObjects",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::634176595009:user/Ec2-S3-Admin"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::karthik-s3-admin-only-2026-001-634176595009-us-west-2-an/*"
    }
  ]
}
```

Policy Explanation

Principal: Specifies the Ec2-S3-Admin IAM user who is allowed to access the bucket.

s3:ListBucket: Allows the Admin user to list the objects inside the bucket.

s3:GetObject: Allows the Admin user to read or download objects.

Bucket ARN: Provides permission at the bucket level.

Object ARN: Provides permission for objects inside the bucket.

Verification

The bucket policy was successfully saved with the correct bucket ARN.

The bucket remained private with Block all public access enabled.

The Ec2-S3-Admin user was granted the required ListBucket and GetObject permissions.

Result

The Admin-only S3 bucket policy was successfully configured and verified.

Evidence

[Screenshot — Private S3 bucket with Block all public access enabled]

[Screenshot — admin-test.txt.txt uploaded in the bucket]

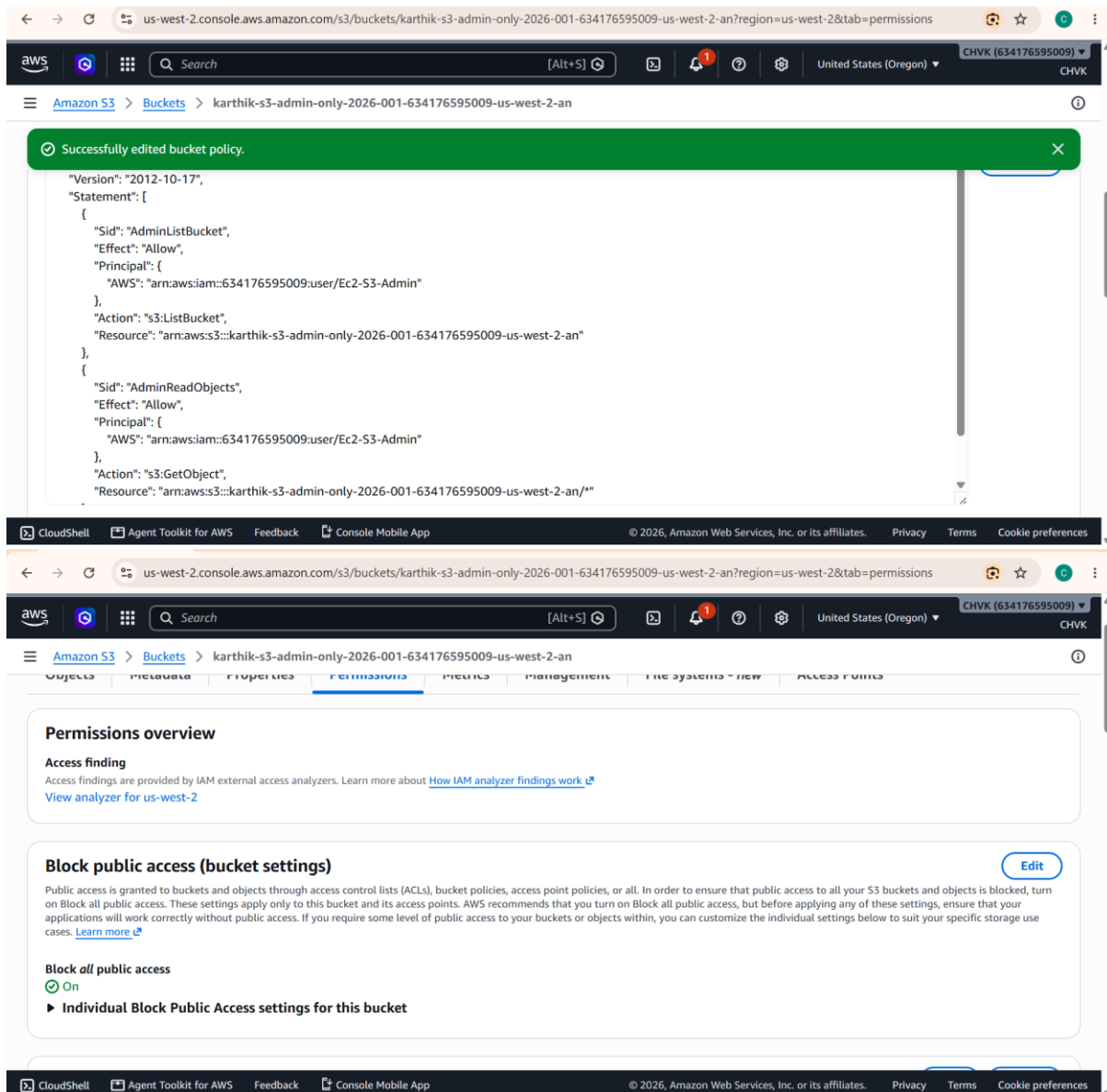
[Screenshot — IAM user Ec2-S3-Admin]

[Screenshot — Final bucket policy successfully saved]

The first screenshot shows the AWS Management Console for the bucket 'karthik-s3-admin-only-2026-001-634176595009-us-west-2-an' in the us-west-2 region. A green notification bar at the top states 'Successfully created bucket "karthik-s3-admin-only-2026-001-634176595009-us-west-2-an"'. The 'Objects' tab is selected, showing 'Objects (0)'. Below the object list, a message states 'No objects. You don't have any objects in this bucket.' with an 'Upload' button.

The second screenshot shows the same bucket with the 'Files and folders' tab selected. A green notification bar at the top states 'Upload succeeded'. Below the notification, a summary shows '1 file, 45.0 B (100.00%)'. The 'Files and folders' table lists one file: 'admin-test.txt.txt' with a status of 'Succeeded'.

Name	Folder	Type	Size	Status	Error
admin-test.txt.txt	-	text/plain	45.0 B	Succeeded	-



- Set up lifecycle policies to automatically transition or delete objects based on specific criteria.

Objective

Configured an S3 Lifecycle Rule to automatically manage objects based on their age and reduce storage costs.

Bucket

karthik-s3-admin-only-2026-001-634176595009-us-west-2-an

Configuration

Lifecycle Rule Name: s3-lifecycle-rules

Status: Enabled

Scope: Entire bucket

Transition Storage Class: Standard-IA

Transition After: 30 days

Current Object Expiration: 365 days

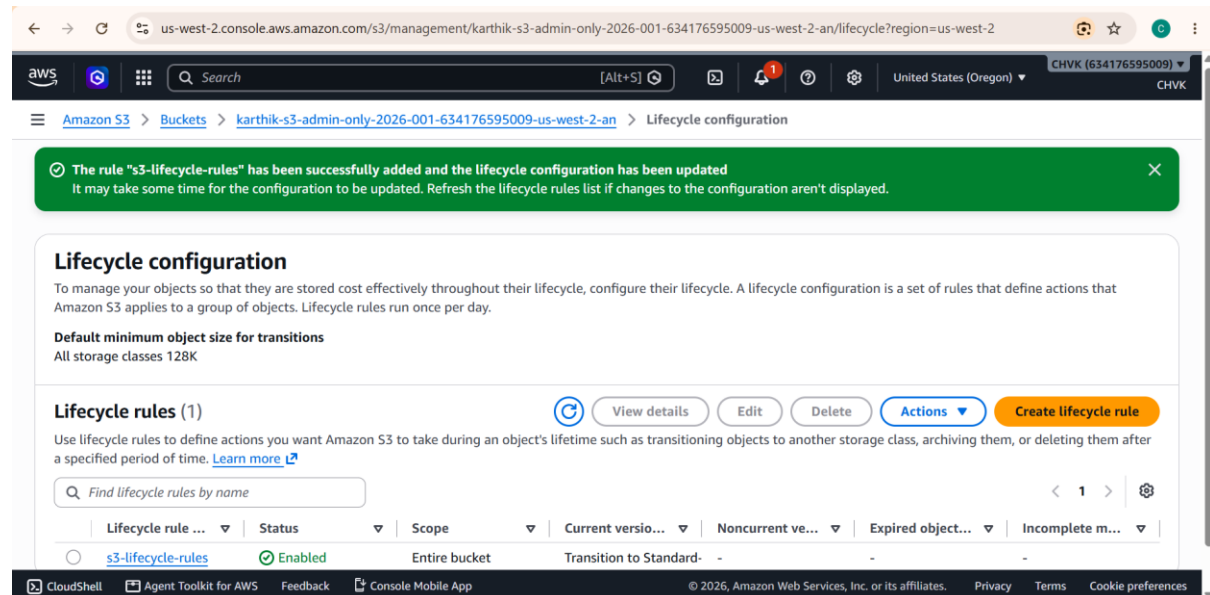
Incomplete Multipart Upload Cleanup: 7 days

Result

The lifecycle rule was successfully created and enabled. Amazon S3 will automatically apply the configured lifecycle actions to objects in the bucket.

Evidence

Screenshot: S3 → Bucket → Management → Lifecycle configuration showing the rule s3-lifecycle-rules with Enabled status.



6. Push some objects to S3 using the AWS CLI.

Objective

Uploaded an object to the S3 bucket using AWS CLI from Git Bash.

Commands Used

```
echo "This file was uploaded using AWS CLI" > cli-test.txt
```

```
cat cli-test.txt
```

```
aws s3 cp cli-test.txt s3://karthik-s3-admin-only-2026-001-634176595009-us-west-2-an/
```

```
aws s3 ls s3://karthik-s3-admin-only-2026-001-634176595009-us-west-2-an/
```

Result

The cli-test.txt file was successfully uploaded to the S3 bucket using AWS CLI.

Evidence

Screenshot showing the successful aws s3 cp upload output and S3 object verification

```

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws --version
aws-cli/2.36.30 Python/3.14.6 Windows/11 exe/AMD64

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws configure
AWS Access Key ID [*****CLMP]: AKIAZHJ664RA44MXQAWH
AWS Secret Access Key [*****KuWT]: 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
Default region name [us-east-1]: us-east-1
Default output format [json]: json

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws sts get-caller-identity
{
  "UserId": "634176595009",
  "Account": "634176595009",
  "Arn": "arn:aws:iam::634176595009:root"
}

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ echo "This file was uploaded using AWS CLI" > cli-test.txt

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ cat cli-test.txt
This file was uploaded using AWS CLI

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws s3 cp cli-test.txt s3://karthik-s3-admin-only-2026-001-634176595009-us-west-2-an/
upload: .\cli-test.txt to s3://karthik-s3-admin-only-2026-001-634176595009-us-west-2-an/cli-test.txt

chvka@karthik MINGW64 ~/OneDrive/Desktop
$

```

7. Write a Bash script to create an S3 bucket.

Objective

Created an Amazon S3 bucket automatically using a Bash script and AWS CLI.

Script

```
#!/bin/bash
```

```
BUCKET_NAME="karthik-s3-bash-2026-001"
```

```
REGION="us-west-2"
```

```
aws s3api create-bucket \
  --bucket "$BUCKET_NAME" \
  --region "$REGION" \
  --create-bucket-configuration LocationConstraint="$REGION"
```

```
echo "S3 bucket created successfully: $BUCKET_NAME"
```

Result

The Bash script successfully created the S3 bucket karthik-s3-bash-2026-001 in the us-west-2 region.

Evidence

Screenshot showing successful script execution and the created bucket in the Amazon S3 console.

```

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ nano create-s3-bucket.sh

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ chmod +x create-s3-bucket.sh

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ ./create-s3-bucket.sh
{
  "Location": "http://karthik-s3-bash-2026-001.s3.amazonaws.com/",
  "BucketArn": "arn:aws:s3:::karthik-s3-bash-2026-001"
}
S3 bucket created successfully: karthik-s3-bash-2026-001

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws s3 ls
2026-09-03 17:04:13 karthik-s3-admin-only-2026-001-634176595009-us-west-2-an
2026-09-03 17:41:29 karthik-s3-bash-2026-001
2026-09-03 16:05:58 karthik-s3-demo-2026-001
2026-09-03 16:35:59 karthik-s3-replica-2026-001

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ |

```

8. Upload a 1 GB file to S3 using the CLI.

Objective

Created a 1 GB test file and uploaded it to Amazon S3 using AWS CLI from Git Bash.

Commands Used

```
dd if=/dev/zero of=1GB-test-file.bin bs=1M count=1024
```

```
ls -lh 1GB-test-file.bin
```

```
aws s3 cp 1GB-test-file.bin s3://karthik-s3-bash-2026-001/
```

```
aws s3 ls s3://karthik-s3-bash-2026-001/
```

Result

A 1 GiB (1,073,741,824 bytes) test file was successfully created and uploaded to the S3 bucket.

The aws s3 ls command confirmed the uploaded object and its size.

Evidence

Screenshot showing the 1 GiB file creation, successful S3 upload, and S3 object verification using AWS CLI.

```
chvka@karthik MINGW64 ~/OneDrive/Desktop
$ dd if=/dev/zero of=1GB-test-file.bin bs=1M count=1024
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied, 3.10414 s, 346 MB/s

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ ls -lh 1GB-test-file.bin
-rw-r--r-- 1 chvka 197609 1.0G Sep  3 17:44 1GB-test-file.bin

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws s3 cp 1GB-test-file.bin s3://karthik-s3-bash-2026-001/
upload: .\1GB-test-file.bin to s3://karthik-s3-bash-2026-001/1GB-test-file.bin

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws s3 ls s3://karthik-s3-bash-2026-001/
2026-09-03 17:45:00 1073741824 1GB-test-file.bin

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ |
```